

Website RAG Customer Support Chatbot

End-to-end n8n pipeline: website ingestion + a retrieval-grounded support agent — technical & process documentation

AI Automation & n8n Workflow Developer



This report documents an n8n automation with two connected parts: a knowledge-base ingestion pipeline that crawls a website with Firecrawl and indexes it into Pinecone, and a Gemini-powered customer support chatbot that answers visitor questions using only that indexed content — a classic Retrieval-Augmented Generation (RAG) setup.

1. Executive Summary

Most support chatbots either hallucinate answers or need every FAQ typed in by hand. This workflow solves both problems: it crawls the business's own website automatically, turns the content into searchable vector embeddings, and gives a Gemini agent a strict rule — answer only from what it finds in that indexed content, and say so honestly when it can't find something. Re-running the ingestion flow keeps the chatbot's knowledge in sync with the live site.

Metric	Detail
Ingestion trigger	Manual trigger — run whenever the website content changes
Chat trigger	n8n chat webhook — public, embeddable widget
Crawling	Firecrawl maps up to 100 URLs from the target site
Chunking	~1,000-character chunks with 200-character overlap, split on Unicode code points
Vector store	Pinecone, cleared and rebuilt on every ingestion run to prevent duplicate chunks
Embeddings & LLM	Google Gemini — same embedding model used for both indexing and retrieval
Grounding rule	The agent must answer business questions from Pinecone only, never from memory or assumption

2. Business Value

- Always-current answers — re-running the ingestion flow re-indexes the live site, so the chatbot never answers from stale, hand-typed FAQs.
- No manual knowledge-base authoring — the entire site's content becomes searchable automatically instead of someone writing and maintaining Q&A pairs.
- Honest about its limits — the agent is instructed to say it couldn't find something and offer a support ticket, rather than guessing an answer.
- Safe by construction — the namespace is fully cleared before every re-index, so the chatbot can never mix outdated and current content.
- Natural conversations — windowed memory keeps the last 10 turns of context so follow-up questions work without the visitor repeating themselves.

3. Architecture Overview

The workflow is two independent flows sharing one Pinecone index: an ingestion pipeline run on demand, and an always-on chat flow that queries whatever the ingestion pipeline has indexed.

Flow	Trigger	Purpose
Knowledge-base Ingestion	Manual trigger, run on demand	Crawls the website, cleans and chunks the content, embeds it, and stores it in Pinecone
Customer Support Chat	Chat webhook, always on	Answers visitor questions using only content retrieved from the indexed knowledge base

4. Ingestion Pipeline — Step by Step

1 **Reset the namespace first**

Before touching the website, the Pinecone index host is resolved and every existing vector in the target namespace is deleted. This runs before crawling so the namespace is rebuilt from scratch every time, guaranteeing no duplicate or stale chunks accumulate across repeated runs.

2 **Map the website**

Firecrawl crawls the configured root URL and returns up to 100 discovered page URLs.

3 **Filter & normalise URLs**

Pages matching blocked keywords (press, gallery, newsletter, and similar sections not worth indexing), XML files, and duplicate URLs (including ones differing only by a trailing slash) are removed, leaving a clean, de-duplicated list of pages to process.

4 **Process one page at a time**

URLs are looped through one at a time rather than all at once, with a short delay between each request so the crawl stays polite to the target server and avoids rate limits.

5 **Fetch & clean the content**

Each page's HTML is fetched, then scripts, styles and tags are stripped, whitespace is collapsed, and any broken Unicode is removed to produce clean readable text. Pages that reduce to fewer than 50 characters are skipped as not worth indexing.

6 **Chunk the text safely**

Usable page text is split into ~1,000-character chunks with a 200-character overlap so context isn't lost at chunk boundaries. Splitting happens on Unicode code points rather than raw characters, so emoji or special characters are never cut in half and corrupted.

7 **Embed & store in Pinecone**

Each chunk is embedded with Google Gemini and stored in Pinecone together with its source URL and original content as metadata, so the chatbot can later cite or reference exactly where an answer came from.

5. Customer Support Chat — Step by Step

1

Receive the message

A public n8n chat trigger accepts visitor messages, ready to be embedded on a website or used standalone.

2

Route to the support agent

The message is handed to a Gemini-powered agent named “Alicia,” configured with a strict system prompt, short-term memory, and the Pinecone knowledge base as a callable tool.

3

Ground every business answer in Pinecone

The agent's system prompt requires it to use the vector store tool for anything about products, services, policies, pricing, FAQs or company/site info — and explicitly forbids answering those topics from memory or assumption. Small talk and greetings skip the tool entirely.

4

Handle the honest failure case

If Pinecone returns nothing useful, or any tool call fails, the agent is instructed to reply with one exact, pre-approved message offering to create a support ticket — never an improvised guess.

5

Keep the conversation natural

A windowed memory node retains the last 10 turns of conversation so the agent can handle natural follow-up questions without the visitor repeating context, while still never storing business information in memory — only a confirmed booking's name, email or phone, if applicable.

6. Node-by-Node Map

Stage	Key nodes
Namespace reset	When clicking 'Execute workflow' → Get Pinecone Index Host → Clear Pinecone Namespace (Prevent Duplicates)
Crawl & filter	Map Website URLs (Firecrawl) → Filter and Normalize URLs
Fetch & clean	Process URLs One by One → Rate Limit Delay → Fetch Page Content → Extract HTML and URL → Clean HTML to Text → Check Text Length
Chunk & index	Split Text into Chunks → Store in Vector Database (+ Document Loader, Embeddings Google Gemini)
Chat	When chat message received → Customer Support Agent (+ Google Gemini Chat Model, Chat Memory, Pinecone Vector Storage tool)

7. Reliability & Security Design

- **Destructive step is deliberate and scoped** — the namespace wipe only ever targets the configured Pinecone namespace, and exists specifically to prevent duplicate chunks from repeated crawls.
- **Strict grounding, no hallucination by design** — the agent's system prompt hard-codes that business information must come from the retrieval tool, with an exact fallback message when nothing relevant is found.
- **Polite, resilient crawling** — pages are processed one at a time with a rate-limit delay, and both the page fetch and Firecrawl map step retry automatically on transient failures without stopping the whole run.
- **Unicode-safe chunking** — splitting by code points (not raw characters) prevents corrupted text from emoji or special characters being fed into the embedding model.
- **Minimal memory footprint** — the agent is explicitly told to store only confirmed contact details after a booking, and never to keep product or company information in conversational memory.

8. Setup Checklist

- Connect credentials: Firecrawl API, Pinecone API, and Google Gemini (PaLM) API.
- Replace `https://example.com/` in Map Website URLs with the real website to index.
- Create a standard dense Pinecone index whose dimension matches the chosen Gemini embedding model, then replace `YOUR_PINECONE_INDEX` and `YOUR_PINECONE_NAMESPACE` everywhere they appear (ingestion and retrieval must use the exact same embedding model).
- Review `blockedKeywords` in Filter and Normalize URLs and adjust them to your site's structure (skip press pages, galleries, newsletters, etc. as needed).
- Run the ingestion flow once manually and confirm vectors appear in Pinecone before publishing the chat trigger.
- Customize the agent's name, greeting, fallback message and system prompt in Customer Support Agent to match your brand, then test retrieval with real visitor-style questions.

This workflow was designed and built end-to-end in n8n as part of an AI automation portfolio, demonstrating a full RAG pipeline: automated web crawling, safe text chunking, vector indexing, and a retrieval-grounded conversational agent.